COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	Oct 15, 2026	**DATE SUBMITTED:**	Oct 19, 2026
NAME:	Isaiah Benedict R. Garbosa Ellaiza C. Vizcayno	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 13 **Parallel Algorithms and Multithreading**

1. PROCEDURE OUTPUT

ILO A & ILO B:

Part 1 : SImple One-Threaded Example

Code

```cpp
#include <iostream>
#include <string>
#include <thread>

void print(int n, const std::string &str){
    std::cout<<"Printing integer: "<<n<<std::endl;
    std::cout<<"Printing string: "<<str<<std::endl;
}

int main(){
    std::thread t1(print, 10, "T.I.P.");
    t1.join();
    return 0;
}
```

Output

Garbosa

```
Printing integer: 10
Printing string: T.I.P.
```

Vizcayno

```
Printing integer: 10
Printing string: T.I.P.
```

Analysis

This simple one-threaded example is similar to using std::cout or print. This is because it only demonstrates a simple one-threaded example. For this program, it features a function pointer and

	its function is called print which prints the integer and string using the given parameters. Then in the main driver, the thread function is used. It is used to execute multiple functions at the same time, but in this case it is used for a simple one thread execution. The thread function creates an object which calls upon the function print, this is called a function pointer, and uses 10 and T.I.P. for the print function's parameters. After using the thread function, join() must be used. This is used to tell the program that the object t1 has not finished executing and to wait for it to finish. Without using join(), the thread would finish its task before the print is completing which would result in an error. Comparing the output from two different devices, there is no difference as this only uses a simple one thread for execution.

Part 2 : Multi- Threaded Example	
Code	

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <thread>

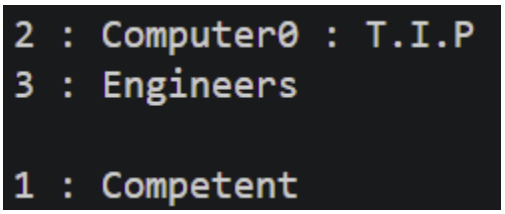
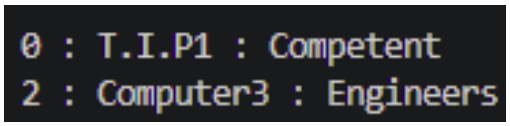
void print(int n, const std::string &str){
    std::string msg = std::to_string(n) + " : " + str;
    std::cout<<msg<<std::endl;
}

int main(){
    std::vector<std::string> s = {
        "T.I.P",
        "Competent",
        "Computer",
        "Engineers",
    };
    std::vector<std::thread> threads;

    for (int i = 0; i<s.size(); i++){
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads){
        th.join();
    }

    return 0;
}
```

Output	Garbosa	Vizcayno
		
Analysis	The program starts by creating a function called print. Its parameters contain an int n and a string. The function first creates a string called msg that displays the integer value, converting it to a string first, then prints a colon separator with the string that was provided in the parameter. After, the string msg is printed. In the main function, it starts by creating a vector s that stores 4 strings which are "T.I.P", "Competent", "Computer", and "Engineers", containing their respective indexes starting with 0. After, it creates a vector for the threads. A for loop is then used to create a thread for each string. The threads.push_back line stores the new thread in the vector while std::thread executes the print function with the current index and string. Another for loop is created. Finally the program uses join to pause the program and wait until the particular thread finishes processing. This ensures that the program does not terminate itself until all of the string is finished processing. After executing, we can notice that the program does not print a constant output. Different outputs can happen since the threads run concurrently, meaning that all of the threads are being processed at the exact same time. So, we can't really predict one output since it becomes a race to what thread finishes first.	

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A	Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.
Question	1. Write a definition of multithreading and its advantages/disadvantages.
	Answers
Garbosa	Multithreading is a programming technique where the program can be divided into multiple threads. These multiple threads can run concurrently or be processed at the same time. So, it can help programs run more effectively and show a much better performance since the program is allowed to execute multiple tasks simultaneously while sharing the same memory space. However, after applying this technique in the previous task from the procedure part, its output can vary. Since threads are being executed at the same time, a some sort of race to what prints first happens. This means that there are no constant outputs, it can change every time the program is executed.
Vizcayno	According to (Mildenberger, 2024), multithreading is a programming concept where it allows multiple threads to be executed concurrently within a single process. Instead of processing one thread at a time, it processes multiple at the same time. This can improve performance, efficiency,

	enhance responsiveness, and utilization of resources. While it is useful, its nature can be unpredictable as it processes multiple threads concurrently, its outputs can vary every time it runs or due to the various devices used to run the program.

Question	2. Rationalize the use of multithreading by providing at least 3 use-cases.
Answers	
Garbosa	Multithreading is very helpful in various cases since it allows multiple tasks to be executed at the same time. An application of this technique that we often see is in web servers. Web servers handle thousands of simultaneous client requests that can not be handled effectively if the requests are processed sequentially. Thus, it uses multithreading to handle these thousands of client requests at the same time without freezing or making other users wait. Another application of multithreading is in web browsers. Most web browsers run each open browser tab, script or media stream on a separate thread. Another thread is used which we can call the network thread that can download images or videos in the background. Lastly, video games use multiple threads to handle main player input and game logic, another thread to render 3D graphics, and a third to calculate physics or AI in the games.
Vizcayno	Multithreading can be applied in numerous ways such as web servers, databases, and gaming engines. In web servers, they process multiple requests from clients as they can be accessed concurrently by users. Databases also use multithreading as they process multiple queries and transactions simultaneously. Lastly, gaming engines also use multithreading as they process the rendering, physic calculations, audio processing, etc, to ensure a smooth gameplay for players.

Question	3. Differentiate between parallelism and concurrency.
Answers	
Garbosa	Parallelism refers to executing multiple tasks simultaneously using multiple processing units. It improves throughput and computation speed since work is divided across processors. Tasks are split into smaller subtasks that run in parallel on separate processors and it mainly focuses on true simultaneous execution. Concurrency on the other hand is managing multiple tasks that make progress during overlapping time periods. Tasks can be interleaved on a single processor or can also run on multiple processors. Tasks execute in overlapping time periods, it makes progress without waiting for others to complete.
Vizcayno	The difference between parallelism and concurrency is that parallelism can refer to multiple cashiers taking orders from multiple customers at the same time while concurrency is when there is only one worker switching from task to task such as taking and preparing orders. Parallelism is taking requests simultaneously and concurrency is overlapping requests to ensure they finish on time.

References	GeeksforGeeks. (2026b, October 6). Difference between Concurrency and Parallelism. GeeksforGeeks. https://www.geeksforgeeks.org/operating-systems/difference-between-concurrency-and-parallelism/ GeeksforGeeks. (2026, August 31). Multithreading in C++. GeeksforGeeks. https://www.geeksforgeeks.org/cpp/multithreading-in-cpp/ Mildenberger, T. (2024, July 4). *Introduction to multithreading*. Contabo Blog. https://contabo.com/blog/introduction-to-multithreading/

Part B	Create C++ Code and show a solution that satisfies the given requirements below.
Code	

```cpp
#include <iostream>
#include <thread>

int globalVar = 0;
void add(int n){
    globalVar += n;
}
int main(){
    std::thread t1(add, 10000);
    std::thread t2(add, 10000);
    std::thread t3(add, 10000);

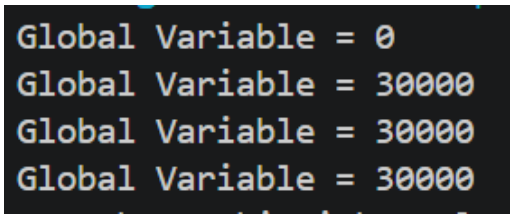
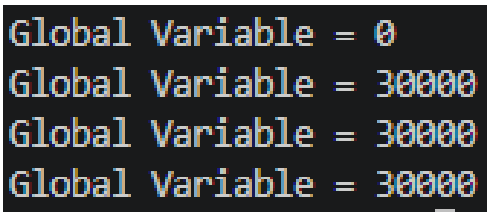
    std::cout<<"Global Variable = "<<globalVar<<std::endl;
    t1.join();

    std::cout<<"Global Variable = "<<globalVar<<std::endl;
    t2.join();

    std::cout<<"Global Variable = "<<globalVar<<std::endl;
    t3.join();

    std::cout<<"Global Variable = "<<globalVar<<std::endl;

    return 0;
}
```

Output	Garbosa	Vizcayno
		
Analysis	The code first sets a global variable called globalVar and initializes it to 0. After, a function called add( ) is created. It receives an integer n and it adds it to the globalVar. Then, three threads are created . Each thread calls the add function with the value 10000, which means that each of the threads attempts to increase the global variable 10000. After, the program displays the value of globalVar four times, calling join() between each display. The join( ) function makes the main thread wait for a thread to finish before continuing. The other threads may still be running while the program waits for a particular thread to finish. That is why, if we look at the output, almost each of the display attempts results in the value of 30000. Furthermore, this is because of how all three threads access and modify the same variable without synchronization. If we look at the first output, it prints 0. This is because the main thread prints the variable before the additions have completed, then after that, all three threads have already finished adding.	

Part C	Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.
Code	**Original Merge Sort:**

```cpp
#include <iostream>
#include <thread>

void merge(int arr[], int left, int middle, int right){
    int i = left;
    int j = middle +1;
    int k = 0;

    int size = right - left + 1;
    int* temp = new int[size];

    while (i <= middle && j <= right){
        if(arr[i] <= arr[j]){
            temp[k] = arr[i];
            i++;
        }
        else{
            temp[k] = arr[j];
            j++;
        }
        k++;
    }

    while (i<=middle){
        temp[k] = arr[i];
        i++;
        k++;
    }

    while(j <= right){
        temp[k] = arr[j];
        j++;
        k++;
    }

    for (int x = 0; x < size; x++){
        arr[left + x] = temp[x];
    }

    delete[] temp;
}
```

```cpp
void mergeSort(int arr[], int left, int right){

    if (left<right){
        int middle = (left + right) / 2;
        mergeSort(arr, left, middle);
        mergeSort(arr, middle+1, right);
        merge(arr, left, middle, right);
    }
}
```

Parallel Merge Sort:

```
void parallelMergerSort(int arr[], int left, int right){
    if (left < right){
        int middle = (left + right) / 2;

        std::thread t1([&](){
            mergeSort(arr, left, middle);
        });
        std::thread t2([&](){
            mergeSort(arr, middle + 1, right);
        });

        t1.join();
        t2.join();

        merge(arr,left,middle,right);
    }
}
```

```
int main(){

    int arr[] = {8,3,6,2,7,1,5,4};
    int size = sizeof(arr)/sizeof(arr[0]);

    std::cout<<"Original Array: ";
    displayArray(arr, size);

    std::cout<<"Multithreaded Merge Sort: ";
    parallelMergerSort(arr, 0, size - 1);
    displayArray(arr, size);

    return 0;
}
```

Output	Garbosa	Vizcayno
	`Original Array: 8 3 6 2 7 1 5 4` `Multithreaded Merge Sort: 1 2 3 4 5 6 7 8`	`Original Array: 8 3 6 2 7 1 5 4` `Multithreaded Merge Sort: 1 2 3 4 5 6 7 8`

Analysis

Our pair picked the algorithm of merge sort to apply multithreading since we believe that it's the algorithm from our past works that suits multithreading the most because of how it solves multiple subarrays. We created a function that incorporates multithreading called parallelMergeSort. Unlike the original merge sort, it uses two threads to sort the separated halves of the array concurrently. This function uses the original merge and merge sort functions. First, the function

	checks if the array contains more than one element using left < right. It then calculates the middle index of the array to split it into two halves. Then, two threads are created called t1 and t2. The thread t1 sorts the left half using merge sort and thread t2 sorts the right half using merge sort as well. The [&] syntax is used here to allow the thread function to access the variables from the surrounding function like the arr, left, right, and middle so that we don't have to pass each variable manually. After, t1.join() and t2.join() is used so that the main thread waits until both of the threads are finished sorting each of their halves. This is important since the halves will need to be sorted first before combining them back together. Finally, the merge() function is called to combine the sorted halves from t1 and t2, creating a sorted array. Overall, multithread helped the function be more efficient since it sorts both the left half and the right half at the same time.

Conclusion - Garbosa
In this activity, we learned about multithreading and parallel algorithms. I learned that by using multithreading, we can run different tasks concurrently or all at the same time. I learned that multithreading is a very useful technique in order for programs to run tasks all at the same time. This makes the program more efficient since it processes multiple tasks all at once. Furthermore, I learned that the join function waits for a specific thread to finish before continuing. However, from the procedure task, I noticed that the output can vary. Multithreading runs tasks concurrently, so if we set threads to a variable, there can be a sort of race to what processes first. So, some of the outputs have different results since threads are processed all at the same time. Overall, I believe I was able to learn a lot from this activity. I believe that multithreading can be useful for different projects in the future as well as for our final project as it can help us provide an efficient running program.

Conclusion - Vizcayno
This activity taught us about multithreading and parallelism. While they sound like the same thing, there is a clear difference between them. Multithreading is when a program uses threads to carry out different tasks independently, while parallelism is when copies of the same program run at the same time on different data. These are useful when fulfilling the requests of many clients at the same time, since there can be a lot of tasks to do. Multithreading and parallel algorithms make it possible for these tasks to be done simultaneously instead of one at a time. We also learned that join() tells the main thread that the other thread hasn't finished yet, so it has to wait for it to finish first. Without it, the main thread could finish before the other thread completes its task. In the activity, we saw how this affects the final output. With multithreading, the output can vary across devices or runtimes because the tasks run at the same time, so you don't know which one will finish first.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The following AI model were used to fix grammar and sentence structures in our analysis and conclusion.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.